

Quantum Circuits

Quantum Algorithms · Lecture 2

Quantum Circuits

Quantum Algorithms · Lecture 2

$$|a\rangle |b\rangle |0\rangle \longmapsto |a\rangle |b\rangle |a \wedge b\rangle$$

By the end of the hour: why a quantum circuit has to compute AND like this, and what it can do that a classical circuit cannot.

Following R. O'Donnell, 15-859BB, Lecture 10.

Plan

1. **Notation, briefly** — states as vectors, gates as matrices, relative phase

Plan

1. **Notation, briefly** — states as vectors, gates as matrices, relative phase
2. **Classical circuits** — Boolean functions, circuit notation, cost

Plan

1. **Notation, briefly** — states as vectors, gates as matrices, relative phase
2. **Classical circuits** — Boolean functions, circuit notation, cost
3. **Quantum circuits** — diagrams vs. algebra, the new gates, interference

Plan

1. **Notation, briefly** — states as vectors, gates as matrices, relative phase
2. **Classical circuits** — Boolean functions, circuit notation, cost
3. **Quantum circuits** — diagrams vs. algebra, the new gates, interference
4. **Converting Classical to Quantum** — why it is not obvious, and how it is done

Part 1

Notation, briefly

One slide of recap, then the piece we did not do last time.

Recap: states and gates

A qubit state is a unit vector in $\mathbb{C}^2$; n qubits live in $\mathbb{C}^{2^n}$, with basis vectors labelled by bit strings:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad |\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1.$$

Recap: states and gates

A qubit state is a unit vector in $\mathbb{C}^2$; n qubits live in $\mathbb{C}^{2^n}$, with basis vectors labelled by bit strings:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad |\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1.$$

Gates are unitary matrices, $U^\dagger U = I$, and applying one is matrix multiplication:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Recap: states and gates

A qubit state is a unit vector in $\mathbb{C}^2$; n qubits live in $\mathbb{C}^{2^n}$, with basis vectors labelled by bit strings:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad |\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1.$$

Gates are unitary matrices, $U^\dagger U = I$, and applying one is matrix multiplication:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Gates side by side multiply; gates stacked in parallel tensor:

$$(A \otimes B)(|\varphi\rangle \otimes |\psi\rangle) = A|\varphi\rangle \otimes B|\psi\rangle$$

Relative phase

Inside a superposition, signs and phases are real data. Compare

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

Relative phase

Inside a superposition, signs and phases are real data. Compare

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

Both are legitimate states. They differ in one sign — the *relative phase* between the two terms.

Relative phase

Inside a superposition, signs and phases are real data. Compare

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

Both are legitimate states. They differ in one sign — the *relative phase* between the two terms. They are also genuinely different states, not two ways of writing one thing:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle \quad \Rightarrow \text{a later gate can tell them apart}$$

Relative phase

Inside a superposition, signs and phases are real data. Compare

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

Both are legitimate states. They differ in one sign — the *relative phase* between the two terms. They are also genuinely different states, not two ways of writing one thing:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle \quad \Rightarrow \text{a later gate can tell them apart}$$

Global phases does nothing.
Relative phases carry information.

Part 2

Classical circuits

Before building a quantum computer, let us agree on what a computer is.

Why start here?

- An honest comparison needs one model.

Why start here?

- An honest comparison needs one model.
- Quantum circuits are the same picture.

Why start here?

- An honest comparison needs one model.
- Quantum circuits are the same picture.
- Quantum algorithms contain classical code (postprocessing).

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

- $\text{AND}(x_1, x_2) = x_1 \wedge x_2$

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

- $\text{AND}(x_1, x_2) = x_1 \wedge x_2$
- $\text{PARITY}(x) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

- $\text{AND}(x_1, x_2) = x_1 \wedge x_2$
- $\text{PARITY}(x) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- $\text{MAJ}(x_1, x_2, x_3) = 1$ iff at least two inputs are 1

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

- $\text{AND}(x_1, x_2) = x_1 \wedge x_2$
- $\text{PARITY}(x) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- $\text{MAJ}(x_1, x_2, x_3) = 1$ iff at least two inputs are 1
- $\text{PRIME}(x) = 1$ iff the n -bit number x is prime

Boolean functions

A classical computational task is a function

$$F : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

Often $m = 1$, which loses nothing important. Examples:

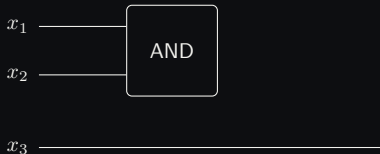
- $\text{AND}(x_1, x_2) = x_1 \wedge x_2$
- $\text{PARITY}(x) = x_1 \oplus x_2 \oplus \cdots \oplus x_n$
- $\text{MAJ}(x_1, x_2, x_3) = 1$ iff at least two inputs are 1
- $\text{PRIME}(x) = 1$ iff the n -bit number x is prime

A function on n bits is exactly a truth table with 2^n rows, so there are

$$2^{2^n}$$

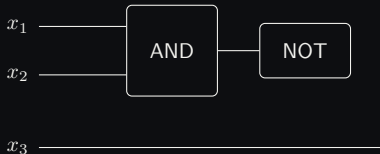
Boolean functions on n bits. For $n = 6$ that is already about 1.8×10^{19} .

Circuit notation



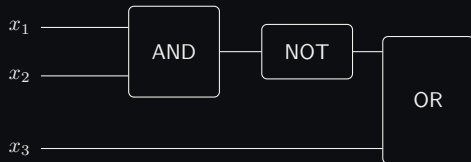
- Wires carry bits left to right. No loops — a circuit is a DAG, not a program with a while loop.

Circuit notation



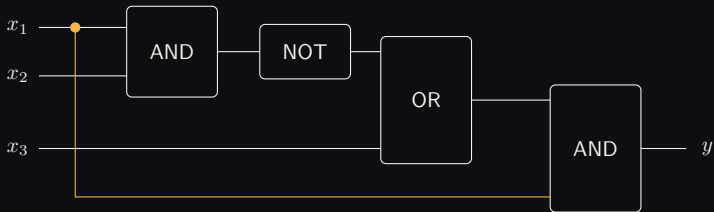
- Wires carry bits left to right. No loops — a circuit is a DAG, not a program with a while loop.
- The gates are AND, OR, NOT. Every Boolean function can be built from these.

Circuit notation



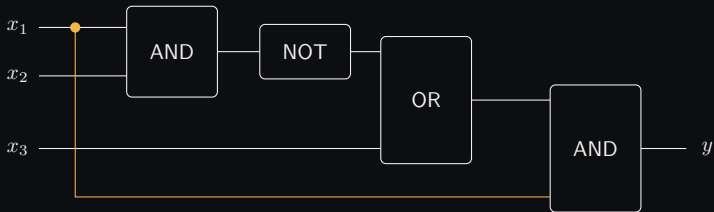
- Wires carry bits left to right. No loops — a circuit is a DAG, not a program with a while loop.
- The gates are AND, OR, NOT. Every Boolean function can be built from these.

Circuit notation



- Wires carry bits left to right. No loops — a circuit is a DAG, not a program with a while loop.
- The gates are AND, OR, NOT. Every Boolean function can be built from these.
- The dot is *fan-out*: one bit duplicated and used twice. Remember this one.

Circuit notation



- Wires carry bits left to right. No loops — a circuit is a DAG, not a program with a while loop.
- The gates are AND, OR, NOT. Every Boolean function can be built from these.
- The dot is *fan-out*: one bit duplicated and used twice. Remember this one.

Read the function straight off the picture: $y = (\neg(x_1 \wedge x_2) \vee x_3) \wedge x_1$.

Circuits vs. the code you actually write

Python `def F(x):`

`# loops, ifs, memory`

`return y`

Circuits vs. the code you actually write

Python `def F(x):`

`# loops, ifs, memory`

`return y` One program, every input

length.

Circuits vs. the code you actually write

Python `def F(x):`

`# loops, ifs, memory`

`return y` One program, every input

length.

Circuits

One circuit C_n per input length: a *family* $\{C_n\}_{n \geq 1}$.

No loops, no memory — everything unrolled in space instead of time.

Circuits vs. the code you actually write

Python `def F(x):`

`# loops, ifs, memory`

`return y` One program, every input

length.

Circuits

One circuit C_n per input length: a *family* $\{C_n\}_{n \geq 1}$.

No loops, no memory — everything unrolled in space instead of time.

A question about complexity

How do we measure the *complexity* of a program? Of a circuit?

Cost

The measure is the **number of gates**, as a function of n :

efficient $\equiv$ gate count is $O(n^k)$ for some constant k .

Cost

The measure is the **number of gates**, as a function of n :

efficient $\equiv$ gate count is $O(n^k)$ for some constant k .

Shannon, 1937. Two facts, both by counting:

Cost

The measure is the **number of gates**, as a function of n :

efficient $\equiv$ gate count is $O(n^k)$ for some constant k .

Shannon, 1937. Two facts, both by counting:

- Every $F : \{0, 1\}^n \rightarrow \{0, 1\}$ has a circuit with $O(2^n/n)$ gates.

Cost

The measure is the **number of gates**, as a function of n :

efficient $\equiv$ gate count is $O(n^k)$ for some constant k .

Shannon, 1937. Two facts, both by counting:

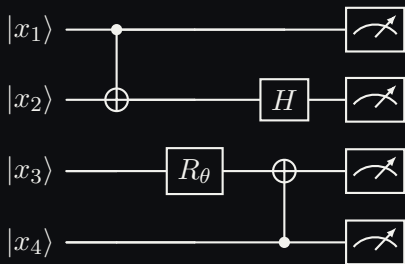
- Every $F : \{0,1\}^n \rightarrow \{0,1\}$ has a circuit with $O(2^n/n)$ gates.
- Almost every F requires $\Omega(2^n/n)$ gates. There are 2^{2^n} functions and far fewer small circuits, so most functions cannot have one.

Part 3

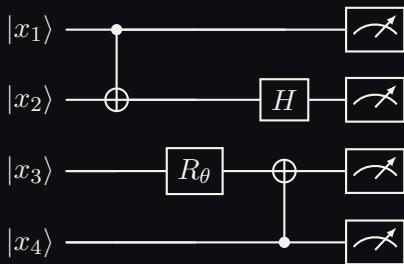
Quantum circuits

Same picture, different algebra.

The model

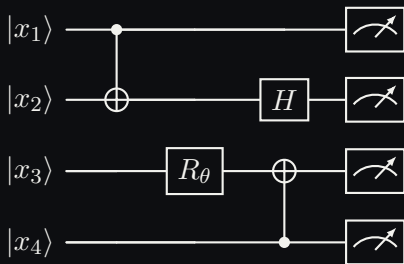


The model



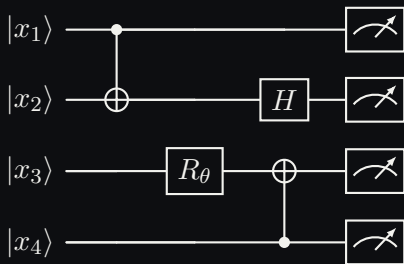
- **Input.** A classical string $x \in \{0, 1\}^n$, loaded as the basis state $|x\rangle$.

The model



- **Input.** A classical string $x \in \{0, 1\}^n$, loaded as the basis state $|x\rangle$.
- **Gates.** A sequence of unitaries, each touching one or two qubits.

The model



- **Input.** A classical string $x \in \{0, 1\}^n$, loaded as the basis state $|x\rangle$.
- **Gates.** A sequence of unitaries, each touching one or two qubits.
- **Readout.** At the end you look at the qubits and get bits out.

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

- X permutes the basis states: the classical NOT.

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

- X permutes the basis states: the classical NOT.
- Z and R_θ leave $|0\rangle$ alone and re-phase $|1\rangle$. On a basis state that does nothing observable; inside a superposition it changes the relative phase.

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

- X permutes the basis states: the classical NOT.
- Z and R_θ leave $|0\rangle$ alone and re-phase $|1\rangle$. On a basis state that does nothing observable; inside a superposition it changes the relative phase.
- H maps basis states to superpositions, and back again.

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

- X permutes the basis states: the classical NOT.
- Z and R_θ leave $|0\rangle$ alone and re-phase $|1\rangle$. On a basis state that does nothing observable; inside a superposition it changes the relative phase.
- H maps basis states to superpositions, and back again.

Acting on qubit j of n means applying $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ to the whole 2^n -dimensional space.

The gates: one qubit

A one-qubit gate is a unitary 2×2 matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad R_\theta = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$$

- X permutes the basis states: the classical NOT.
- Z and R_θ leave $|0\rangle$ alone and re-phase $|1\rangle$. On a basis state that does nothing observable; inside a superposition it changes the relative phase.
- H maps basis states to superpositions, and back again.

Acting on qubit j of n means applying $I \otimes \cdots \otimes U \otimes \cdots \otimes I$ to the whole 2^n -dimensional space.

One-qubit gates can never entangle anything.
For that we need a gate where one wire depends on another.

The gates: controlled unitaries

Take any unitary U . Its *controlled* version cU (or *controlled- U*) adds one wire:

$$cU |0\rangle |\psi\rangle = |0\rangle |\psi\rangle, \quad cU |1\rangle |\psi\rangle = |1\rangle U |\psi\rangle.$$

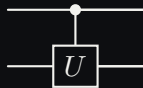
The gates: controlled unitaries

Take any unitary U . Its *controlled* version cU (or *controlled- U*) adds one wire:

$$cU |0\rangle |\psi\rangle = |0\rangle |\psi\rangle, \quad cU |1\rangle |\psi\rangle = |1\rangle U |\psi\rangle.$$

$$cU = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}$$

unitary whenever U is



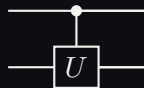
filled dot = control

The gates: controlled unitaries

Take any unitary U . Its *controlled* version cU (or *controlled- U*) adds one wire:

$$cU |0\rangle |\psi\rangle = |0\rangle |\psi\rangle, \quad cU |1\rangle |\psi\rangle = |1\rangle U |\psi\rangle.$$

$$cU = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix} \quad \text{unitary whenever } U \text{ is}$$



filled dot = control

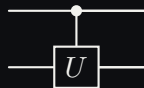
$$cX = \text{CNOT} : |a\rangle |b\rangle \mapsto |a\rangle |b \oplus a\rangle, \quad cZ, \quad cR_\theta, \quad ccX = \text{Toffoli}$$

The gates: controlled unitaries

Take any unitary U . Its *controlled* version cU (or *controlled- U*) adds one wire:

$$cU |0\rangle |\psi\rangle = |0\rangle |\psi\rangle, \quad cU |1\rangle |\psi\rangle = |1\rangle U |\psi\rangle.$$

$$cU = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix} \quad \text{unitary whenever } U \text{ is}$$

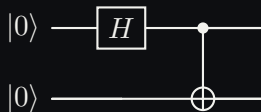


filled dot = control

$$cX = \text{CNOT} : |a\rangle |b\rangle \mapsto |a\rangle |b \oplus a\rangle, \quad cZ, \quad cR_\theta, \quad ccX = \text{Toffoli}$$

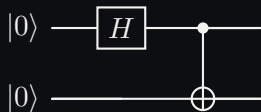
Nothing says the control must be $|0\rangle$ or $|1\rangle$: by linearity,
$$cU(\alpha |0\rangle + \beta |1\rangle) |\psi\rangle = \alpha |0\rangle |\psi\rangle + \beta |1\rangle U |\psi\rangle.$$

Diagram or algebra: the same object, written backwards



time flows $\rightarrow$

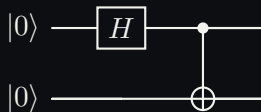
Diagram or algebra: the same object, written backwards



time flows $\rightarrow$

$$|\psi\rangle = \text{CNOT}(H \otimes I) |00\rangle$$

Diagram or algebra: the same object, written backwards

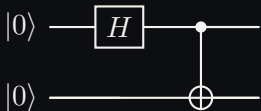


time flows $\rightarrow$

$$|\psi\rangle = \text{CNOT}(H \otimes I) |00\rangle$$

matrices apply $\leftarrow$

Diagram or algebra: the same object, written backwards



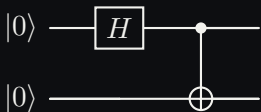
time flows $\rightarrow$

$$|\psi\rangle = \text{CNOT}(H \otimes I) |00\rangle$$

matrices apply $\leftarrow$

- Gates *side by side* in the diagram $\Rightarrow$ matrix **product**, written right to left.

Diagram or algebra: the same object, written backwards



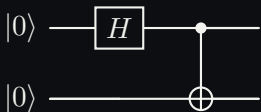
time flows $\rightarrow$

$$|\psi\rangle = \text{CNOT}(H \otimes I) |00\rangle$$

matrices apply $\leftarrow$

- Gates *side by side* in the diagram $\Rightarrow$ matrix **product**, written right to left.
- Gates *stacked vertically* $\Rightarrow$ **tensor product**, top factor on the left. A gate on one wire is $I \otimes U \otimes I$ on the whole space.

Diagram or algebra: the same object, written backwards



$$|\psi\rangle = \text{CNOT}(H \otimes I) |00\rangle$$

time flows $\rightarrow$

matrices apply $\leftarrow$

- Gates *side by side* in the diagram $\Rightarrow$ matrix **product**, written right to left.
- Gates *stacked vertically* $\Rightarrow$ **tensor product**, top factor on the left. A gate on one wire is $I \otimes U \otimes I$ on the whole space.
- A circuit on n qubits is one $2^n \times 2^n$ unitary $U = U_T \cdots U_2 U_1$. We never write that matrix down. The circuit *is* the compressed description of it — exactly as a classical circuit compresses a 2^n -row truth table.

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

$$(H \otimes I) |00\rangle = (H |0\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

$$(H \otimes I) |00\rangle = (H |0\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

$$\text{CNOT} |00\rangle = |00\rangle, \quad \text{CNOT} |10\rangle = |11\rangle$$

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

$$(H \otimes I) |00\rangle = (H |0\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

$$\text{CNOT} |00\rangle = |00\rangle, \quad \text{CNOT} |10\rangle = |11\rangle$$

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

$$(H \otimes I) |00\rangle = (H |0\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

$$\text{CNOT} |00\rangle = |00\rangle, \quad \text{CNOT} |10\rangle = |11\rangle$$

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Working it out

$$|\psi\rangle = \text{CNOT} (H \otimes I) |00\rangle$$

$$(H \otimes I) |00\rangle = (H |0\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

$$\text{CNOT} |00\rangle = |00\rangle, \quad \text{CNOT} |10\rangle = |11\rangle$$

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

This state is not $|\varphi\rangle \otimes |\psi\rangle$ for any single-qubit states — it is entangled. Two gates were enough to leave the world of separate, independent bits.

What is actually new

- Superposition.

What is actually new

- Superposition.
- Entanglement.

What is actually new

- Superposition.
- Entanglement.
- Interference.

Interference, in the smallest possible example

$$H |0\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Interference, in the smallest possible example

$$H |0\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Apply H again, to each term separately:

$$H\left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) = \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) + \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle\right)$$

Interference, in the smallest possible example

$$H |0\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Apply H again, to each term separately:

$$\begin{aligned} H\left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) &= \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) + \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle\right) \\ &= \underbrace{\left(\frac{1}{2} + \frac{1}{2}\right)}_{=1} |0\rangle + \underbrace{\left(\frac{1}{2} - \frac{1}{2}\right)}_{=0} |1\rangle = |0\rangle \end{aligned}$$

Interference, in the smallest possible example

$$H |0\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Apply H again, to each term separately:

$$\begin{aligned} H\left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) &= \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) + \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle\right) \\ &= \underbrace{\left(\frac{1}{2} + \frac{1}{2}\right)}_{=1} |0\rangle + \underbrace{\left(\frac{1}{2} - \frac{1}{2}\right)}_{=0} |1\rangle = |0\rangle \end{aligned}$$

The minus sign came from the single -1 in H . Two routes lead to $|1\rangle$; they arrive with opposite signs and annihilate. Two routes lead to $|0\rangle$; they reinforce.

Interference, in the smallest possible example

$$H |0\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Apply H again, to each term separately:

$$\begin{aligned} H\left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) &= \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle\right) + \frac{1}{\sqrt{2}} \left(\frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle\right) \\ &= \underbrace{\left(\frac{1}{2} + \frac{1}{2}\right)}_{=1} |0\rangle + \underbrace{\left(\frac{1}{2} - \frac{1}{2}\right)}_{=0} |1\rangle = |0\rangle \end{aligned}$$

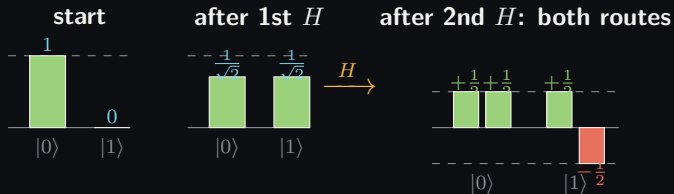
The minus sign came from the single -1 in H . Two routes lead to $|1\rangle$; they arrive with opposite signs and annihilate. Two routes lead to $|0\rangle$; they reinforce.

$H^2 = I$ is not an accident of matrix algebra. It is destructive interference.

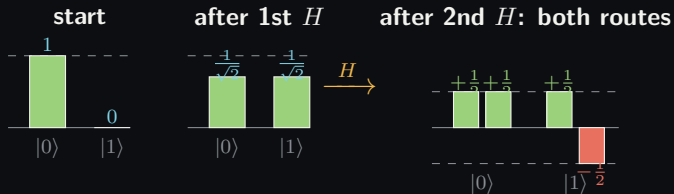
The two paths, as amplitude bars



The two paths, as amplitude bars



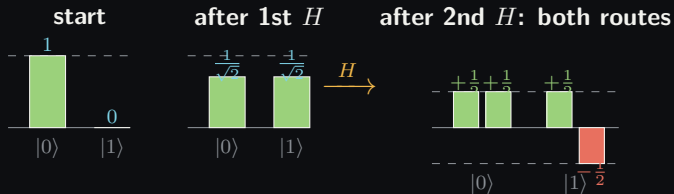
The two paths, as amplitude bars



$$\frac{1}{2} + \frac{1}{2} = 1$$

$$\frac{1}{2} - \frac{1}{2} = 0$$

The two paths, as amplitude bars

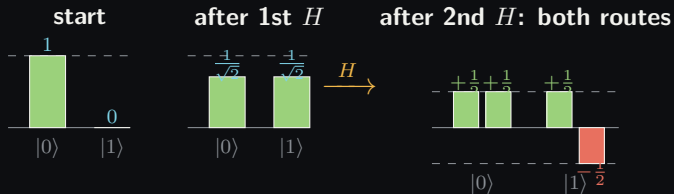


$$\frac{1}{2} + \frac{1}{2} = 1$$

$$\frac{1}{2} - \frac{1}{2} = 0$$

The amplitude of an outcome is the sum over all routes to it, each route contributing the product of its amplitudes along the way.

The two paths, as amplitude bars



$$\frac{1}{2} + \frac{1}{2} = 1$$

$$\frac{1}{2} - \frac{1}{2} = 0$$

The amplitude of an outcome is the sum over all routes to it, each route contributing the product of its amplitudes along the way.

Designing a quantum algorithm means arranging the phases so that the routes to wrong answers cancel.

Part 4

Turning a classical circuit into a quantum one

It had better be possible. It is not obvious that it is.

The obstruction

A quantum gate is a unitary, and unitaries are invertible:

$$U^\dagger U = I \implies U^{-1} = U^\dagger.$$

The obstruction

A quantum gate is a unitary, and unitaries are invertible:

$$U^\dagger U = I \implies U^{-1} = U^\dagger.$$

So every quantum gate is reversible: given the output you can always recover the input.

The obstruction

A quantum gate is a unitary, and unitaries are invertible:

$$U^\dagger U = I \implies U^{-1} = U^\dagger.$$

So every quantum gate is reversible: given the output you can always recover the input. AND is not:

$$00, 01, 10 \mapsto 0 \quad \text{which input was it?}$$

The obstruction

A quantum gate is a unitary, and unitaries are invertible:

$$U^\dagger U = I \implies U^{-1} = U^\dagger.$$

So **every quantum gate is reversible**: given the output you can always recover the input. AND is not:

$$00, 01, 10 \longmapsto 0 \quad \text{which input was it?}$$

AND *erases information*, and so does OR. A quantum circuit cannot contain either of them as drawn.

The obstruction

A quantum gate is a unitary, and unitaries are invertible:

$$U^\dagger U = I \implies U^{-1} = U^\dagger.$$

So **every quantum gate is reversible**: given the output you can always recover the input. AND is not:

$$00, 01, 10 \mapsto 0 \quad \text{which input was it?}$$

AND *erases information*, and so does OR. A quantum circuit cannot contain either of them as drawn.

So: can every classical computation be rearranged to throw nothing away?

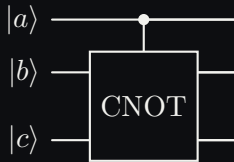
The Toffoli gate, as a controlled gate

No need to invent a new gate. Build it the way every controlled gate gets built: take a unitary, add a control wire — and here the unitary being controlled is CNOT itself.

The Toffoli gate, as a controlled gate

No need to invent a new gate. Build it the way every controlled gate gets built: take a unitary, add a control wire — and here the unitary being controlled is CNOT itself.

$$ccX = c(\text{CNOT}) = \begin{pmatrix} I_4 & 0 \\ 0 & \text{CNOT} \end{pmatrix}$$

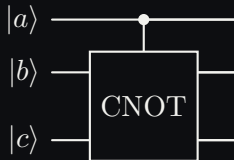


The Toffoli gate, as a controlled gate

No need to invent a new gate. Build it the way every controlled gate gets built: take a unitary, add a control wire — and here the unitary being controlled is CNOT itself.

$$ccX = c(\text{CNOT}) = \begin{pmatrix} I_4 & 0 \\ 0 & \text{CNOT} \end{pmatrix}$$

Same recipe as cU from two slides ago, just with $U = \text{CNOT}$: an 8×8 matrix built from two 4×4 blocks.

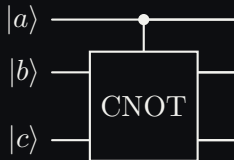


The Toffoli gate, as a controlled gate

No need to invent a new gate. Build it the way every controlled gate gets built: take a unitary, add a control wire — and here the unitary being controlled is CNOT itself.

$$ccX = c(\text{CNOT}) = \begin{pmatrix} I_4 & 0 \\ 0 & \text{CNOT} \end{pmatrix}$$

Same recipe as cU from two slides ago, just with $U = \text{CNOT}$: an 8×8 matrix built from two 4×4 blocks.



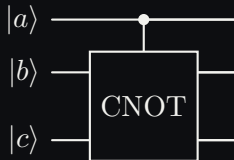
$$c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle \otimes \text{CNOT}^a |b\rangle |c\rangle$$

The Toffoli gate, as a controlled gate

No need to invent a new gate. Build it the way every controlled gate gets built: take a unitary, add a control wire — and here the unitary being controlled is CNOT itself.

$$ccX = c(\text{CNOT}) = \begin{pmatrix} I_4 & 0 \\ 0 & \text{CNOT} \end{pmatrix}$$

Same recipe as cU from two slides ago, just with $U = \text{CNOT}$: an 8×8 matrix built from two 4×4 blocks.



$$c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle \otimes \text{CNOT}^a |b\rangle |c\rangle$$

a decides whether the CNOT on b, c fires at all.

Expanding $c(\text{CNOT})$

- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.

Expanding $c(\text{CNOT})$

- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.
- If $a = 1$: the CNOT fires, flipping c exactly when $b = 1$:
 $c(\text{CNOT}) |1\rangle |b\rangle |c\rangle = |1\rangle |b\rangle |c \oplus b\rangle$.

Expanding $c(\text{CNOT})$

- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.
- If $a = 1$: the CNOT fires, flipping c exactly when $b = 1$:
 $c(\text{CNOT}) |1\rangle |b\rangle |c\rangle = |1\rangle |b\rangle |c \oplus b\rangle$.
- Both cases at once, one formula: $c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle |b\rangle |c \oplus (a \wedge b)\rangle$.

Expanding $c(\text{CNOT})$

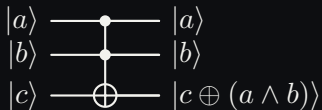
- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.
- If $a = 1$: the CNOT fires, flipping c exactly when $b = 1$:
 $c(\text{CNOT}) |1\rangle |b\rangle |c\rangle = |1\rangle |b\rangle |c \oplus b\rangle$.
- Both cases at once, one formula: $c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle |b\rangle |c \oplus (a \wedge b)\rangle$.

Flips only when $a = 1$ *and* $b = 1$ — exactly the Toffoli / ccX we wanted, drawn the usual way with two control dots and one target:

Expanding $c(\text{CNOT})$

- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.
- If $a = 1$: the CNOT fires, flipping c exactly when $b = 1$:
 $c(\text{CNOT}) |1\rangle |b\rangle |c\rangle = |1\rangle |b\rangle |c \oplus b\rangle$.
- Both cases at once, one formula: $c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle |b\rangle |c \oplus (a \wedge b)\rangle$.

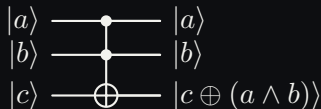
Flips only when $a = 1$ *and* $b = 1$ — exactly the Toffoli / ccX we wanted, drawn the usual way with two control dots and one target:



Expanding $c(\text{CNOT})$

- If $a = 0$: the CNOT does not fire, so $c(\text{CNOT}) |0\rangle |b\rangle |c\rangle = |0\rangle |b\rangle |c\rangle$.
- If $a = 1$: the CNOT fires, flipping c exactly when $b = 1$:
 $c(\text{CNOT}) |1\rangle |b\rangle |c\rangle = |1\rangle |b\rangle |c \oplus b\rangle$.
- Both cases at once, one formula: $c(\text{CNOT}) |a\rangle |b\rangle |c\rangle = |a\rangle |b\rangle |c \oplus (a \wedge b)\rangle$.

Flips only when $a = 1$ *and* $b = 1$ — exactly the Toffoli / ccX we wanted, drawn the usual way with two control dots and one target:



It permutes the eight basis states (swapping $|110\rangle \leftrightarrow |111\rangle$, fixing the rest): unitary, and self-inverse.

The classical gates, reversibly

Set $c = 0$ in the Toffoli: the bottom wire comes out holding $a \wedge b$.

NOT



already reversible

The classical gates, reversibly

Set $c = 0$ in the Toffoli: the bottom wire comes out holding $a \wedge b$.

NOT



already reversible

AND



Toffoli, spare $|0\rangle$

The classical gates, reversibly

Set $c = 0$ in the Toffoli: the bottom wire comes out holding $a \wedge b$.

NOT



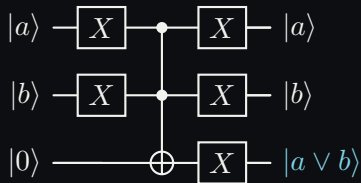
already reversible

AND



Toffoli, spare $|0\rangle$

OR



De Morgan: $a \vee b = \neg(\neg a \wedge \neg b)$

The classical gates, reversibly

Set $c = 0$ in the Toffoli: the bottom wire comes out holding $a \wedge b$.

NOT



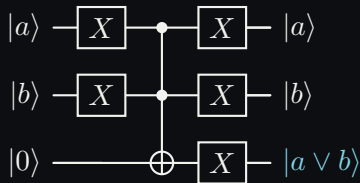
already reversible

AND



Toffoli, spare $|0\rangle$

OR



De Morgan: $a \vee b = \neg(\neg a \wedge \neg b)$

Negate the inputs, reuse the same Toffoli, negate back: four X gates and one Toffoli.

The classical gates, reversibly

Set $c = 0$ in the Toffoli: the bottom wire comes out holding $a \wedge b$.

NOT



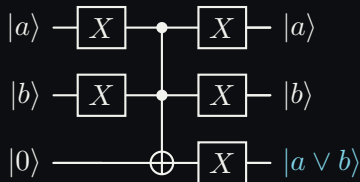
already reversible

AND



Toffoli, spare $|0\rangle$

OR



De Morgan: $a \vee b = \neg(\neg a \wedge \neg b)$

Negate the inputs, reuse the same Toffoli, negate back: four X gates and one Toffoli.

The NOT and AND gates for a complete gate set, and now we can represent every classical boolean function by composing them together. (At the cost of leftover "ancilla" qubits.)

Every classical circuit is a quantum circuit

- If $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *bijection* — reversible, as a classical computation — it is already a unitary:

$$U_F |x\rangle = |F(x)\rangle$$

just permutes the basis states. This is exactly why the Toffoli gate is unitary.

Every classical circuit is a quantum circuit

- If $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *bijection* — reversible, as a classical computation — it is already a unitary:

$$U_F |x\rangle = |F(x)\rangle$$

just permutes the basis states. This is exactly why the Toffoli gate is unitary.

- Most functions are not bijections. AND sends three different inputs to 0: nothing to invert.

Every classical circuit is a quantum circuit

- If $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *bijection* — reversible, as a classical computation — it is already a unitary:

$$U_F |x\rangle = |F(x)\rangle$$

just permutes the basis states. This is exactly why the Toffoli gate is unitary.

- Most functions are not bijections. AND sends three different inputs to 0: nothing to invert.

The general fix, for *any* $F : \{0, 1\}^n \rightarrow \{0, 1\}^m$, reversible or not:

$$U_F |x\rangle |y\rangle = |x\rangle |y \oplus F(x)\rangle$$

Every classical circuit is a quantum circuit

- If $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *bijection* — reversible, as a classical computation — it is already a unitary:

$$U_F |x\rangle = |F(x)\rangle$$

just permutes the basis states. This is exactly why the Toffoli gate is unitary.

- Most functions are not bijections. AND sends three different inputs to 0: nothing to invert.

The general fix, for *any* $F : \{0, 1\}^n \rightarrow \{0, 1\}^m$, reversible or not:

$$U_F |x\rangle |y\rangle = |x\rangle |y \oplus F(x)\rangle$$

Why unitary: apply it twice and y comes back, since $y \oplus F(x) \oplus F(x) = y$. A map that is its own inverse permutes the basis states, and every permutation of basis states is unitary.

Every classical circuit is a quantum circuit

- If $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *bijection* — reversible, as a classical computation — it is already a unitary:

$$U_F |x\rangle = |F(x)\rangle$$

just permutes the basis states. This is exactly why the Toffoli gate is unitary.

- Most functions are not bijections. AND sends three different inputs to 0: nothing to invert.

The general fix, for *any* $F : \{0, 1\}^n \rightarrow \{0, 1\}^m$, reversible or not:

$$U_F |x\rangle |y\rangle = |x\rangle |y \oplus F(x)\rangle$$

Why unitary: apply it twice and y comes back, since $y \oplus F(x) \oplus F(x) = y$. A map that is its own inverse permutes the basis states, and every permutation of basis states is unitary.

$$y = 0 : \quad U_F |x\rangle |0\rangle = |x\rangle |F(x)\rangle$$

Fan-out, at last

The gate set aside earlier: duplicate a bit onto a fresh ancilla.

Fan-out, at last

The gate set aside earlier: duplicate a bit onto a fresh ancilla.



Fan-out, at last

The gate set aside earlier: duplicate a bit onto a fresh ancilla.



Check both basis states:

$$\text{CNOT } |0\rangle |0\rangle = |0\rangle |0\rangle, \quad \text{CNOT } |1\rangle |0\rangle = |1\rangle |1\rangle$$

Fan-out, at last

The gate set aside earlier: duplicate a bit onto a fresh ancilla.



Check both basis states:

$$\text{CNOT } |0\rangle |0\rangle = |0\rangle |0\rangle, \quad \text{CNOT } |1\rangle |0\rangle = |1\rangle |1\rangle$$

Tempting conclusion: this circuit duplicates whatever state sits on the first wire.
That's a trap, a was a *bit*, 0 or 1, not an arbitrary qubit.

The same circuit, on a superposition

Feed the same circuit $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ instead of a basis state:

$$\text{CNOT } |+\rangle |0\rangle = \text{CNOT } \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

The same circuit, on a superposition

Feed the same circuit $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ instead of a basis state:

$$\text{CNOT } |+\rangle |0\rangle = \text{CNOT } \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Copying would have produced $|+\rangle \otimes |+\rangle$ instead:

$$|+\rangle |+\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

The same circuit, on a superposition

Feed the same circuit $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ instead of a basis state:

$$\text{CNOT } |+\rangle |0\rangle = \text{CNOT } \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Copying would have produced $|+\rangle \otimes |+\rangle$ instead:

$$|+\rangle |+\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

Not the same state. The circuit entangled the two wires — it did not copy $|+\rangle$ onto the second one.

No-cloning

Could some *other* unitary copy an arbitrary qubit? Suppose $U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle$ held for every state $|\psi\rangle$.

No-cloning

Could some *other* unitary copy an arbitrary qubit? Suppose $U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle$ held for every state $|\psi\rangle$. It would in particular have to do this on the two basis states:

$$U |0\rangle |0\rangle = |0\rangle |0\rangle, \quad U |1\rangle |0\rangle = |1\rangle |1\rangle$$

No-cloning

Could some *other* unitary copy an arbitrary qubit? Suppose $U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle$ held for every state $|\psi\rangle$. It would in particular have to do this on the two basis states:

$$U |0\rangle |0\rangle = |0\rangle |0\rangle, \quad U |1\rangle |0\rangle = |1\rangle |1\rangle$$

Linearity then fixes what U does to $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, no choice left:

$$U |\psi\rangle |0\rangle = \alpha |0\rangle |0\rangle + \beta |1\rangle |1\rangle$$

No-cloning

Could some *other* unitary copy an arbitrary qubit? Suppose $U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle$ held for every state $|\psi\rangle$. It would in particular have to do this on the two basis states:

$$U |0\rangle |0\rangle = |0\rangle |0\rangle, \quad U |1\rangle |0\rangle = |1\rangle |1\rangle$$

Linearity then fixes what U does to $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, no choice left:

$$U |\psi\rangle |0\rangle = \alpha |0\rangle |0\rangle + \beta |1\rangle |1\rangle$$

But copying $|\psi\rangle$ means producing $|\psi\rangle |\psi\rangle$ instead:

$$|\psi\rangle |\psi\rangle = \alpha^2 |00\rangle + \alpha\beta |01\rangle + \alpha\beta |10\rangle + \beta^2 |11\rangle$$

No-cloning

Could some *other* unitary copy an arbitrary qubit? Suppose $U |\psi\rangle |0\rangle = |\psi\rangle |\psi\rangle$ held for every state $|\psi\rangle$. It would in particular have to do this on the two basis states:

$$U |0\rangle |0\rangle = |0\rangle |0\rangle, \quad U |1\rangle |0\rangle = |1\rangle |1\rangle$$

Linearity then fixes what U does to $|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$, no choice left:

$$U |\psi\rangle |0\rangle = \alpha |0\rangle |0\rangle + \beta |1\rangle |1\rangle$$

But copying $|\psi\rangle$ means producing $|\psi\rangle |\psi\rangle$ instead:

$$|\psi\rangle |\psi\rangle = \alpha^2 |00\rangle + \alpha\beta |01\rangle + \alpha\beta |10\rangle + \beta^2 |11\rangle$$

The two agree only when $\alpha\beta = 0$, i.e. $|\psi\rangle$ was already $|0\rangle$ or $|1\rangle$. No unitary clones an unknown qubit state — *no-cloning* (Wootters–Zurek, Dieks, 1982).

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...
reversible	no	always

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...
reversible	no	always
AND, OR, fan-out	yes	yes, with ancillas

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...
reversible	no	always
AND, OR, fan-out	yes	yes, with ancillas
superposed inputs	no	yes

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...
reversible	no	always
AND, OR, fan-out	yes	yes, with ancillas
superposed inputs	no	yes
contributions cancel	no	yes

What we gained

	Classical circuit	Quantum circuit
a wire holds	one bit	one amplitude per string
gates	AND, OR, NOT, fan-out	H , R_θ , CNOT, Toffoli, ...
reversible	no	always
AND, OR, fan-out	yes	yes, with ancillas
superposed inputs	no	yes
contributions cancel	no	yes

The first four rows: quantum circuits contain classical ones. The last two: the extra functionality.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.
- Quantum circuits are the same diagrams with unitary gates, read left to right while the matrix product reads right to left.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.
- Quantum circuits are the same diagrams with unitary gates, read left to right while the matrix product reads right to left.
- The new ingredients are superposition on a wire and relative phase. H twice shows what phase is for: cancellation.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.
- Quantum circuits are the same diagrams with unitary gates, read left to right while the matrix product reads right to left.
- The new ingredients are superposition on a wire and relative phase. H twice shows what phase is for: cancellation.
- Controlled unitaries give us CNOT and Toffoli, and with one ancilla each they reproduce NOT, AND and OR reversibly.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.
- Quantum circuits are the same diagrams with unitary gates, read left to right while the matrix product reads right to left.
- The new ingredients are superposition on a wire and relative phase. H twice shows what phase is for: cancellation.
- Controlled unitaries give us CNOT and Toffoli, and with one ancilla each they reproduce NOT, AND and OR reversibly.
- So every classical circuit is a quantum circuit, and quantum circuits can do strictly more.

Recap

- A classical computation is a Boolean function; a circuit is a wiring diagram that compresses its 2^n -row truth table. Cost is gate count.
- Quantum circuits are the same diagrams with unitary gates, read left to right while the matrix product reads right to left.
- The new ingredients are superposition on a wire and relative phase. H twice shows what phase is for: cancellation.
- Controlled unitaries give us CNOT and Toffoli, and with one ancilla each they reproduce NOT, AND and OR reversibly.
- So every classical circuit is a quantum circuit, and quantum circuits can do strictly more.
- Fan-out duplicates a *bit*, not a qubit. No unitary can, by no-cloning.